

Demo Video Plan

Hospital Patient Vital Signs Monitoring — EC8203 Mini-Project

Sep 30, 2026 · @Akila

Overview

Target length: **8 minutes** (inside the brief's 5-10 minute window, with slack for a stumble). The brief asks for one thing above all: "showing the pipeline running end-to-end and the observability results." Everyone talks over their own screen, in their own voice, no slides: a live terminal, browser tabs and dashboards, recorded as one continuous take per person and stitched together.

Segment	Member	Length	Covers
1. Intro, architecture, ingestion	A	~2.5 min	Business question, Lambda decision (one paragraph), architecture diagram, both simulated sources running
2. Speed layer	B	~2.5 min	Spark Structured Streaming, live risk tiers, a real-time alert firing
3. Batch layer, serving, wrap-up	C	~3 min	Airflow DAG, the daily report, the API, limitations, close

Each member's section below is a minute-by-minute script: what to type or click, and a suggested line to say (adapt it to your own words — memorising a script on camera reads worse than a natural paraphrase of the same points). Bold text in the "say" column is the one fact that must survive paraphrasing.

Pre-recording checklist (all three, before anyone hits record)

Do this together on a call, in this order, 30-45 minutes before recording:

- Pick one machine to record on** (whoever has the stack running most reliably). The other two join by screen-share on the same call and talk over their own turn; only one machine's screen is ever recorded at a time.
- Fresh start.** On that machine: `make reset` (or `docker compose down -v` then delete `data/state`, `data/landing`, `data/lake`, `data/checkpoints`, `data/reports`, `data/alerts`, `data/ground_truth`). This restarts simulated time at day 1 so the story

(P001's sepsis episode, the occult patients' labs) plays out predictably on camera.

3. **Bring the stack up early.** `docker compose up -d --build`, then wait. Building from scratch takes several minutes; do this *before* the call so nobody watches a progress bar on camera. Confirm with `docker compose ps` that all 17 services are `Up / healthy`.
4. **Warm it up for about 6-7 minutes** (one to two simulated days) so the first lab file has landed and the speed layer has real windows before Member A starts talking. Check with `python scripts/e2e_smoke.py --wait 60` — all checks should pass.
5. **Open every tab you'll need in advance**, in this order left to right: a terminal, Grafana (`localhost:3000`, Pipeline Health dashboard), Grafana Ward Live dashboard in a second tab, Airflow (`localhost:8080`), API docs (`localhost:8000/docs`), and the architecture diagram (from `Report.pdf / Report.docx` or the live doc). Log into Grafana and Airflow now so no password prompt appears on camera.
6. **Silence notifications** (Slack, email, Windows notifications) and close unrelated tabs/windows before recording.
7. **One dry run, untimed.** Everyone reads their section out loud once, over the real screen, without recording, so hand-offs between members don't have dead air. Note anything that's slower live than expected (in particular: DagFailed and the missing-lab-file demo both take several minutes to fire — see Member B and C's sections for the shortcut).
8. **Agree who edits.** One person owns stitching the three clips together (Section 6, Recording and editing logistics) and shares the final file for the others to sanity-check before submission.

Member A — intro, architecture, ingestion (~2.5 min)

Every command below runs in **one terminal window, on the recording machine, from the project root** (`D:\bigdta\Hospital-Patient-Vital-Signs-Monitoring`), full-screened or large enough to read on the recording. Browser steps use whichever browser is already logged in from the pre-recording checklist.

Time	Where	Command / action	What to show on screen	Say
0:00-0:20	Terminal	<code>docker compose ps</code>	The full list of ~17 containers, all Up / healthy — let it sit on screen a few seconds before you start talking over it	"Hi, I'm [name]. This is our EC8203 mini-project: a hospital ward monitoring pipeline. The question it answers is: which patients look concerning right now, and how do yesterday's lab results change that picture? "

Time	Where	Command / action	What to show on screen	Say
0:20-0:55	PDF viewer or browser	Open <code>Report.pdf</code> , scroll to the architecture diagram page (Section 3, "Architecture diagram")	The whole diagram on screen, full width; point the cursor at the two boxes labelled "Spark Structured Streaming" and "Airflow DAG" as you say their names	"We built this as a Lambda architecture : a Spark speed layer for seconds-level alerts, and an Airflow batch layer for the daily lab file, because those two things need genuinely different latency and replay guarantees. The two views meet in one PostgreSQL database."
0:55-1:15	Terminal	<code>docker compose ps</code> again	Same list, briefly, to bridge back from the diagram to the running system	"Everything you just saw runs as this one Docker Compose stack."

Time	Where	Command / action	What to show on screen	Say
1:15-1:35	Terminal	<pre>docker compose exec kafka /opt/kafka/bin/kafka- topics.sh -- bootstrap-server kafka:9092 --describe --topic vitals.raw</pre>	The <code>PartitionCount:</code> 3 line in the output — pause and let the cursor rest under it	"On the ingestion side, a Python script simulates 20 bedside monitors, one reading every two seconds, into a Kafka topic with 3 partitions keyed by patient , so each patient's readings stay in order."

Time	Where	Command / action	What to show on screen	Say
1:35-2:00	Terminal	<code>docker compose logs - -tail 8 vitals- simulator then docker compose logs --tail 8 lab-generator</code>	The JSON log lines scrolling by — no need to read them aloud, just let the audience see real structured logs	"The producer is idempotent and we deliberately inject faults — nulls, duplicates, late events — to prove the pipeline handles them; you'll see that in a minute. A second script drops one lab-results file per simulated day, standing in for the pathology lab."

Time	Where	Command / action	What to show on screen	Say
2:00-2:30	Browser tab: Grafana, <code>localhost:3000</code> , Pipeline Health dashboard	Switch tabs, no command	The top row of stat tiles (firing alerts, throughput, simulated day)	"Everything here is observable: structured logs, Prometheus metrics, thirteen alert rules. Here's the live pipeline-health view — I'll hand over to [Member B] to show the speed layer that's actually driving these numbers."

If something's not on screen in time: skip straight to the Kafka describe command and the Grafana dashboard; the topic/lab-file detail can be said without the terminal output visible.

Member B — speed layer and a live alert (~2.5 min)

Member B takes over the **same screen-share, same machine**, straight after Member A (do not stop or reset the stack). All terminal commands run in the same terminal, from the project root.

Time	Where	Command / action	What to show on screen	Say
0:00-0:25	Browser tab: Grafana Ward Live Monitoring dashboard (<code>localhost:3000</code>)	Switch tabs, no command	The risk-tier stat tiles across the top, then the "Patients by risk" table — hover/point at the highest-risk row	"This is the speed layer's output: every patient's current risk tier, updated within seconds of each reading. It's a Spark Structured Streaming job — two queries reading the same Kafka topic, one scoring every reading, one aggregating into 2-minute windows for trends."

Time	Where	Command / action	What to show on screen	Say
0:25-0:50	Terminal	<pre>docker compose exec postgres psql -U hospital -c "SELECT patient_id, risk_tier, news_score, lab_risk_points, trend_flag, last_reading_at FROM patient_status ORDER BY total_score DESC LIMIT 5"</pre>	The printed table — point the cursor at the top row's <code>risk_tier</code> , <code>news_score</code> and <code>lab_risk_points</code> columns as you name them	"Here's that table directly: [name the top patient] is at [tier] right now, with a NEWS score of [X] and [Y] lab points already factored in — that's the score our shared scoring module computes identically in both the speed and batch layers."
0:50-1:00	Terminal	<pre>docker compose stop spark-streaming</pre>	The command echoing back <code>Stopping... Stopped</code>	"Let me show this is actually monitored, not just a diagram. I'm stopping the speed layer right now."
1:00-1:35	Browser tab: Prometheus Alerts page (localhost:9090/alerts)	Switch tabs, refresh once or twice while talking	Watch <code>StreamingQueryStopped</code> and <code>ConsumerLagHigh</code> change from nothing to red/"firing" in the list — point the cursor at the alert name the moment it turns red	"Within a minute, StreamingQueryStopped and then ConsumerLagHigh fire — Prometheus noticed the job is gone and that Kafka lag is climbing, and both alerts route through Alertmanager to a webhook that logs them permanently."
1:35-2:05	Terminal	<pre>docker compose start spark-streaming, wait ~15s, then re-run the same psql query from row 2, plus docker compose exec postgres psql -U hospital -c "SELECT count(*), count(DISTINCT alert_id) FROM alerts"</pre>	The two counts in the last query's output being equal	"And restarting it: it resumes from its checkpoint. Both queries stay idempotent — rerunning this query shows the same alert count as distinct alert IDs, so nothing double-fired during the restart."
2:05-2:30	Browser tab: Grafana Pipeline Health dashboard	Switch tabs, no command	The "Injected faults by type" and "Valid vs rejected (DLQ) readings" panels	"The simulator also injects bad data on purpose — nulls, duplicates, late events — and this panel is that being caught and routed to a dead-letter topic instead of corrupting anything. Over to [Member C] for what happens once a day, on the batch side."

Timing note: the alert takes 45s-70s to go from stopped to firing — don't rush the explanation in row 3/4, the alert needs that time anyway. If it's still pending when you must move on, say so honestly ("it's about to fire") rather than waiting in silence; the report has the full timing evidence.

Member C — batch layer, serving API, wrap-up (~3 min)

Member C takes over the **same screen-share, same machine**, straight after Member B. Terminal commands run in the same terminal, from the project root; browser steps use new tabs alongside the ones already open.

Time	Where	Command / action	What to show on screen	Say
0:00-0:30	Browser tab: Airflow (<code>localhost:8080</code>)	Open the <code>daily_lab_risk_report</code> DAG, click Graph view	The full task graph, then click into one green/completed run so every task box shows green	"Once a day, this DAG runs: it waits for the lab file, validates every row — quarantining anything bad rather than failing — loads it, scores the lab risk, recomputes yesterday's vitals from our Parquet lake, and builds the daily report. Here's a run that's already completed today."
0:30-0:55	Same Airflow tab	No new command; optionally mention <code>make replay-day DAY=N</code> without running it	The task list, all green	"Every one of these steps is idempotent — we can replay any past day with one command and get byte-identical results, which we use for testing and which is also our disaster-recovery story."

Time	Where	Command / action	What to show on screen	Say
0:55-1:30	New browser tab: <code>localhost:8000/api/reports/risk/latest/html</code>	Navigate to that URL	The rendered report table — scroll to and point the cursor at a row where <code>risk_before_labs</code> and <code>risk_after_labs</code> differ	"This is the actual deliverable the assignment asks for — patients ranked by risk, with a before-labs and after-labs column . [Point at a row where the tier changed] This patient looked fine on vitals alone; the lab file moved them up because of an abnormal result — that's the business question, answered."
1:30-2:00	New browser tab: <code>localhost:8000/docs</code>	Find <code>GET /api/ward/summary</code> , click Try it out , then Execute	The JSON response — scroll to show fields from both layers (for example <code>active_alerts</code> and <code>batch</code>) in the same payload	"The serving API is where both layers meet: this one response has the live risk tiers from Spark and the latest batch report and reconciliation number from Airflow, together."
2:00-2:25	Back to Grafana Pipeline Health tab	Switch tabs, no command	The "Speed vs batch discrepancy" panel, if visible; otherwise skip the visual and just say the number	"We also measure how much the two layers disagree — it's stayed under 1% across a 43-day soak test, an order of magnitude inside our alert threshold, which is the evidence for choosing a Lambda architecture over Kappa in the first place."

Time	Where	Command / action	What to show on screen	Say
2:25-3:00	Face camera, no screen needed (or leave the last dashboard on screen)	—	—	"A few honest limitations: the scoring is a simplified NEWS2, not clinically validated; everything runs on one laptop with a single broker and no auth, which we'd change at production scale — all written up in the report. Thanks for watching."

If Airflow has no completed run yet (warm-up was too short): stay in the terminal and run `docker compose exec airflow-scheduler airflow dags list-runs -d daily_lab_risk_report -o table`, which lists past runs even without the UI, and narrate row 1 over that output instead.

Recording and editing logistics

Tooling. Any screen recorder with a microphone track works: OBS Studio (free, records locally, no time limit) is the safest choice; a Zoom/Teams call with cloud recording is an easy fallback if everyone is remote — have the presenter share their screen and record the call.

One continuous stack, three recordings. Because all three members work from the same running stack (Section 2), record in this order without resetting in between:

1. Member A records their 2.5 minutes first, screen-sharing the machine from the checklist.
2. Immediately after, without touching `docker compose down`, Member B takes over the screen-share and records their segment — the alert demo in Section 4 needs the stack to still be warm and running.
3. Member C records last, once Airflow has had time to complete at least one run.

If recording on separate calls/days instead: re-run the whole pre-recording checklist (Section 2) each time, and accept that the exact patient/day numbers on screen will differ between segments — that's fine, the narration doesn't depend on specific numbers.

Stitching. Concatenate the three clips in order (A, B, C) with any simple video editor (DaVinci Resolve, CapCut, or even a phone editing app all handle a straight cut with no transitions needed). Trim dead air at the start/end of each clip; a one-second cross-fade between segments looks cleaner than a hard cut but is optional.

Captions/title card (optional but cheap). A 3-second opening title card with the project name and team names, and burned-in captions if your editor supports auto-captioning, both help a grader skimming multiple submissions.

Export. 1080p, under 500 MB, MP4 (H.264) is accepted everywhere. Watch the final cut once, full length, before calling it done — check audio levels are consistent across all three segments.

Final checklist before submitting

- ☐ Total length is inside 5-10 minutes
- ☐ The pipeline is shown running end-to-end: ingestion → speed layer → batch layer → serving/dashboards
- ☐ At least one alert is shown actually firing live, not just described
- ☐ The daily report's before-labs/after-labs comparison is visible on screen at some point
- ☐ Audio is clear and consistent across all three segments
- ☐ All three members' voices are heard
- ☐ File is MP4, under 500 MB, and plays back correctly after export
- ☐ Uploaded/linked per the module's submission instructions alongside `Report.pdf`